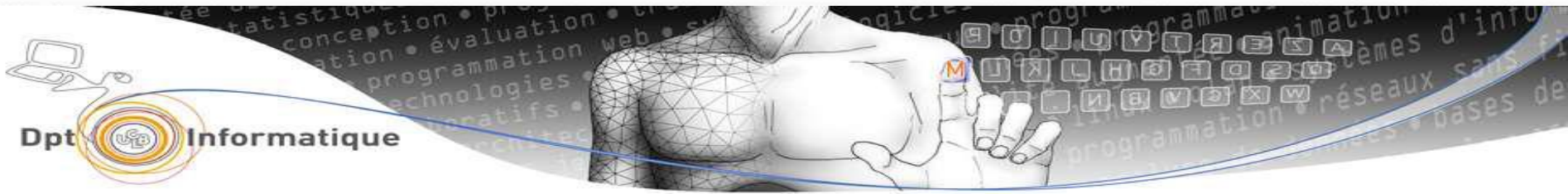
UPSSA
UNIVERSITÉ PRIVÉE DE LA SANTÉ ET DES SCIENCES D'AGADIR
الجامعة الخاصة للصحة و العلوم لأكادير

PROGRAMMATION ORIENTÉE OBJET EN JAVA

Said CHARFI
Maitre de conférences habilité
charfisaid@gmail.com/s.charfi@uiz.ac.ma

Filière : Génie Informatique



Présentation de JAVA

Bases de Java

Historique

- 1991, chez SUN est créé le langage OAK (pour interface entre appareils ménagers et ordinateurs)
- 1994, Oak se tourne vers Internet et devient Java
- 1995, Java est intégré à Netscape
- 1998, Version 2 de Java
- 2000, Version 1.3 de Java
- 2002, Version 1.4 de Java
- 2004, Java 5
- 2011, Java JDK 7
- 2014, Java 8 (LTS)
- 2018, Java 11 (LTS)
- 2021, Java 17 (LTS)
- 2023, Java 21 (LTS)
- 2025, Java 25 (LTS, dernière version à support long terme)
- 2026, Java 27 (dernière version, septembre 2026)

Présentation

- Java est un langage récent qui s'est répandu de façon étonnamment rapide

- Car : Java est portable, non dépendant de l'environnement d'implémentation (de la machine)

- Car : la syntaxe de Java est très proche du langage C++ (et donc de C)…

Présentation

- Car : Java est robuste
  - Gestion de la mémoire par un système de ramasse - miettes (Garbage Collector)
  - Typage fort
- Car : Il existe un grand nombre de classes sûres et commentées
- Car Java est gratuit …

Présentation

- Java est "vraiment" Orienté Objet :
  tout est objet
- Les programmes java sont multi plate-forme
- Langage haute sécurité
- Java est utilisé pour créer des programmes et des Applets
- Java fournit un code (ByteCode) interprété par une machine virtuelle
  - (+) Garantit la portabilité
  - (-) Ralentit l'exécution

Télécharger Java

- Sur le site (www.oracle.com), Oracle propose les différents Kits de Développement Java (JDK)
- https://www.oracle.com/technetwork/java/javase/downloads/index.html

- Sur Internet, grand nombre de sources et de documentation
  [Ex : "Thinking in Java" de B. Eckel]
  http://sd.blackball.lv/library/Thinking_in_Java_4th_edition.pdf



- Programmer en Java (2ème édition)
  - Auteur : Claude Delannoy
  - Éditeur : Eyrolles
  - Edition : 2002 - 661 pages - ISBN : 2212111193



- Java en action
  - Auteur : Ian F. Darwin
  - Éditeur : O'Reilly
  - Edition : 2002 - 836 pages - ISBN : 2841772039



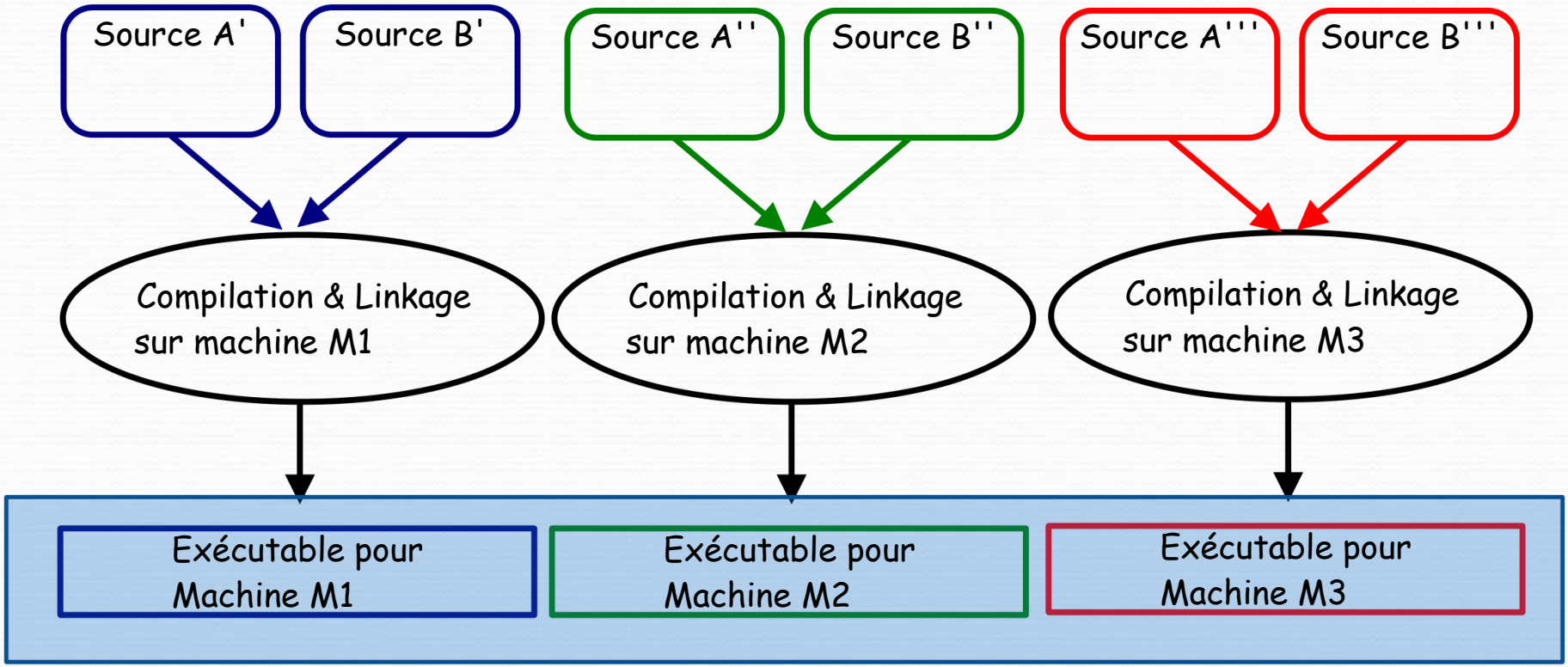
- Apprendre Java et C++ en parallèle
  - Auteur : Jean-Bernard Boichat
  - Éditeur : Eyrolles
  - Edition : 2003 - 742 pages - ISBN : 2212113277

Les concepts de JAVA

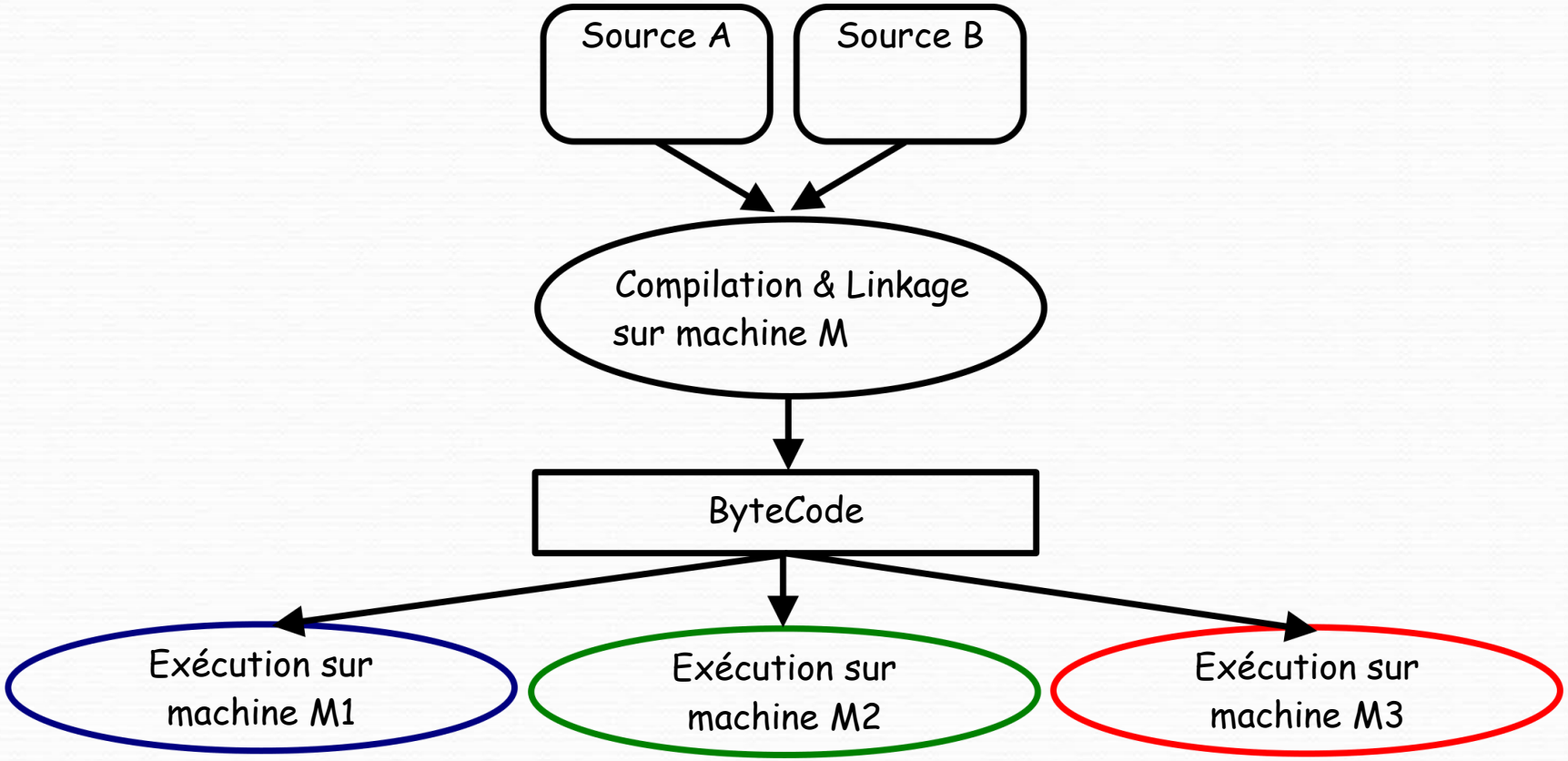
La Compilation & JAVA

- Le principal atout de JAVA concerne sa portabilité.
- Celle-ci est possible par la notion de ByteCode.
- JAVA ne produit pas un code natif, directement exécutable.
- JAVA produit un code intermédiaire, le ByteCode, interprétable par une machine virtuelle.

La Compilation "classique"



La Compilation Java



Interprétation par machine virtuelle (Java Virtual Machine)
spécifique au système d'exploitation

Code natif vs ByteCode

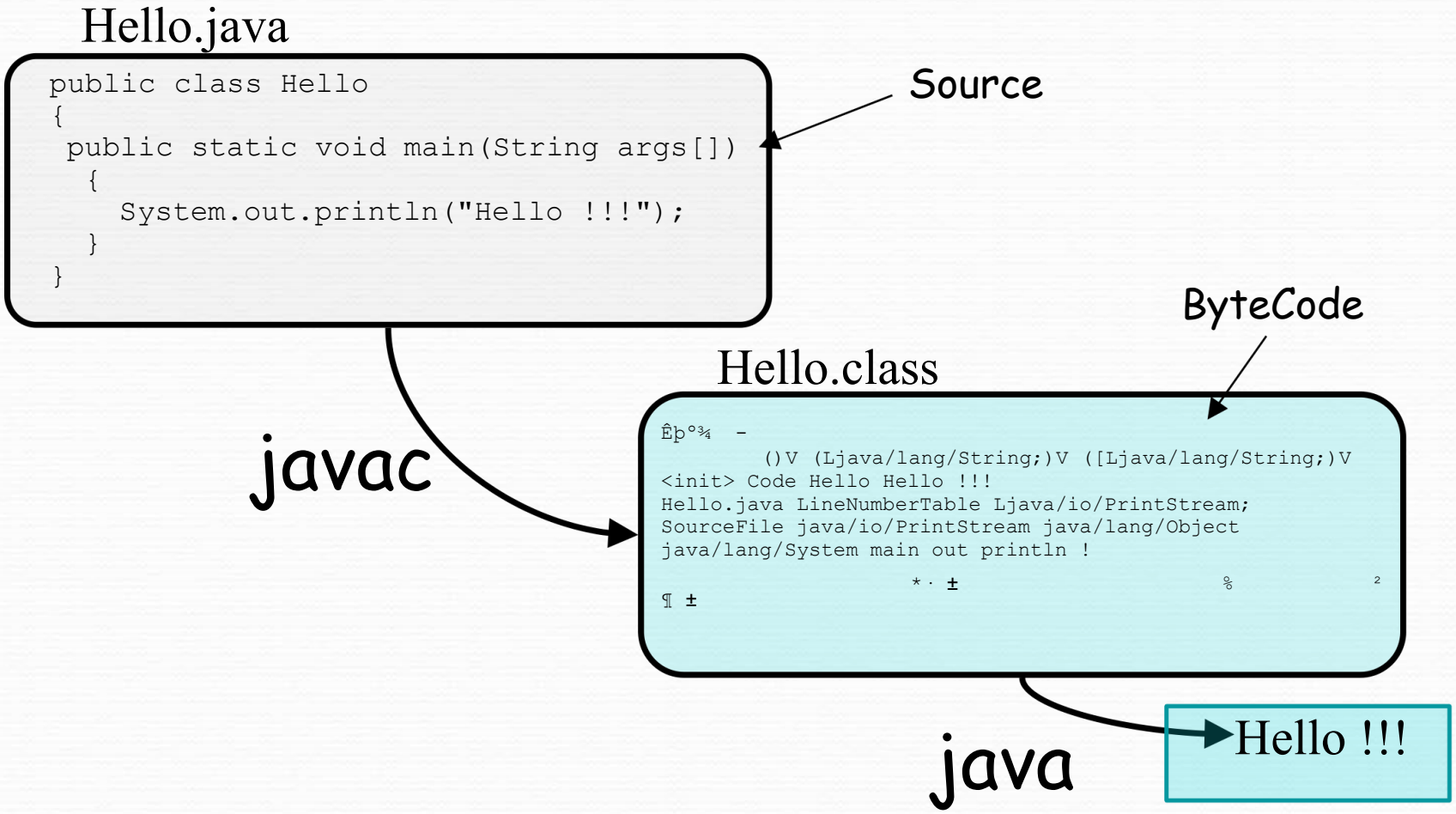
- (+) rapidité d'exécution
- (-) source pas toujours portable
- (-) recompiler pour changer de système

- (+) source portable
- (+) Bytecode portable
- (-) moins rapide car interprété

Source portable

- Les types primitifs sont totalement spécifiés.
- Ils ne dépendent pas de l'architecture de la machine :
  - Les entiers, réels, etc… sont toujours définis sur le même nombre de bits.
- Génération du ByteCode par le compilateur javac

Exemple de compilation



Ramasse Miettes

- En JAVA, tout est objet. Il faut donc créer explicitement chaque objet manipulé.
- Cependant, il est inutile de les détruire. Le Garbage Collector (ramasse miettes) s'en charge.
- Dès qu'un objet n'est plus utilisé (sortie de boucle, ...), il est détruit.

Eléments de base

Tout est Objet (ou presque...)

- Tous les éléments en JAVA dérivent de la classe Object.
- Tous sauf les primitives (8 en tout) représentant des types simples.
- A chaque type de primitive correspond une classe d'objet.

Les primitives : des types simples

Nom	*Taille*	*Minimum*	*Maximum*	*Classe*
boolean	1 bit	false	true	Boolean
char	16 bits	Unicode 0	Unicode $2^{16}-1$	Character
byte	8 bits	-128	127	Byte
short	16 bits	-2^{15}	$2^{15}-1$	Short
int	32 bits	-2^{31}	$2^{31}-1$	Integer
long	64 bits	-2^{63}	$2^{63}-1$	Long
float	32 bits	$(+/-)1.4\ 10^{-45}$	$(+/-)3.4\ 10^{38}$	Float
double	64 bits	$(+/-)4.9\ 10^{-324}$	$(+/-)1.8\ 10^{308}$	Double
void	-	-	-	Void

Les primitives : Casting

Casting en Java

1. Casting implicite — sur-casting

Un sur-casting peut être effectué d'une manière implicite.

```
int a = 6;       // int : 32 bits
long b;         // long : 64 bits
b = a;           // Casting implicite
```

2. Casting explicite

Un casting explicite consiste à faire précéder la valeur par le nouveau type entre parenthèses :

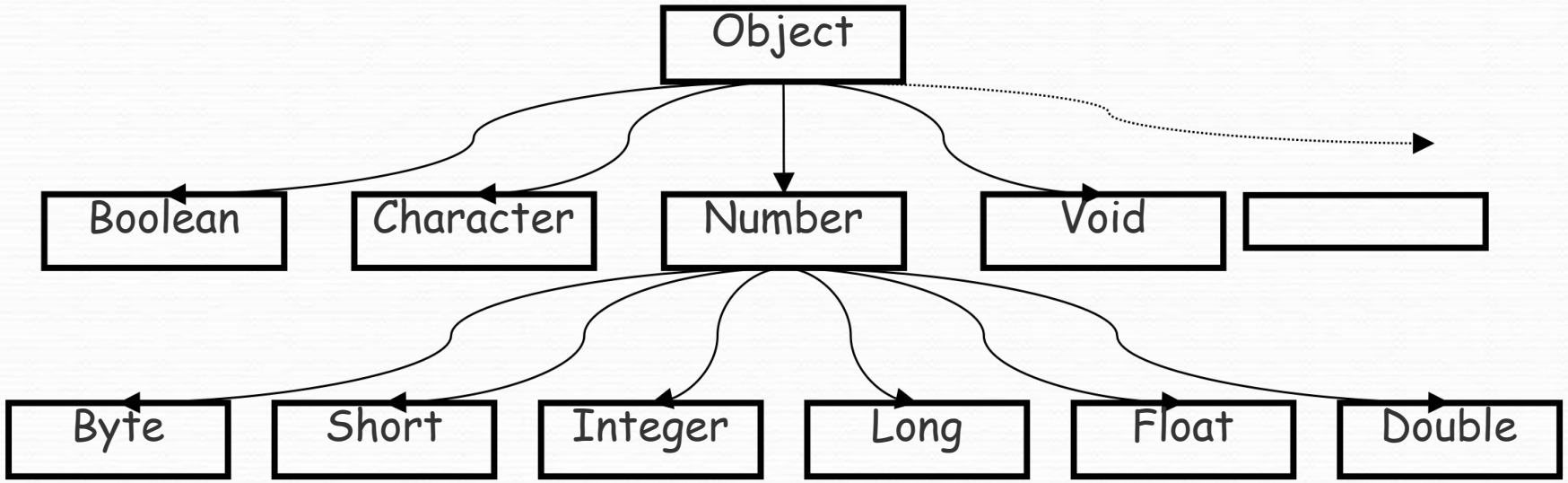
```
long b;
int a = 6;
b = (long) a;
```

Exemples :

```
float a = (float) 5.5;
double c = (double) a;
int d = 8;
byte f = (byte) d;
```

Les objets simples

- Tout objet Java hérite de la classe Object :



Primitives <-> Objets

- Passer d'une primitive à l'objet correspondant :

```
int i = 5;
Integer obj = new Integer (i);
```

- Dans la classe Number sont définies les méthodes de conversion intValue, doubleValue, floatValue, ...

```
int j = obj.intValue();
```

- Dans la classe Object est définie la méthode toString retournant la valeur d'un objet sous forme de chaîne.

Les opérateurs

- Arithmétiques :

  pour tout nombre x, on peut écrire :
  x++; x--; ++x; --x; x+=2; x-=2; x*=2; x/=2;
  x%=2 (x = x modulo 2)

  <, <=, >, >=, == égalité

  != : différence; ! : négation

- Logiques : pour tout booléen x et y,

  non x ⇔!x
  x et y ⇔x & y ou x && y *(test économe)*
  x ou y ⇔x | y ou x || y *(test économe)*
  x ou exclusif y ⇔x ^ y

Priorité des opérateurs

N°	Catégorie	Opérateurs
1	Opérateurs postfix	`expr++ expr--`
2	Opérateurs unaires	`++expr --expr +expr -expr ~ !`
3	Multiplication, division, modulo	`* / %`
4	Addition et soustraction	`+ -`
5	Décalage	`<< >> >>>`
6	Comparaison	`< > <= >= instanceof`
7	Égalité	`== !=`
8	OU exclusif	`^`
9	ET	`&`
10	OU	`|`
11	ET logique	`&&`
12	OU logique	`||`
13	Ternaire	`? :`
14	Assignement	`= += -= *= /= %= ^= |= <<= >>= >>>=`
15	Arrow	`->`

Entrées clavier

Scanner : méthodes de lecture

Méthode	Retour	Rôle
`nextLine()`	String	Lit une ligne complète
`next()`	String	Lit un mot (jusqu'à l'espace)
`nextInt()`	int	Lit un entier
`nextDouble()`	double	Lit un réel
`nextBoolean()`	boolean	Lit true ou false
`hasNextInt()`	boolean	Teste si le prochain mot est un entier

```
Scanner sc = new Scanner(System.in);
System.out.print("Nom : ");
String nom = sc.nextLine();
System.out.print("Age : ");
int age = sc.nextInt();
System.out.println(nom + " a " + age + " ans");
sc.close();
```

Piège : nextInt() puis nextLine()

```
int age = sc.nextInt();          // lit 25, laisse le
retour à la ligne
String nom = sc.nextLine();    // renvoie "" (fin de la
ligne précédente)
// Solution 1 : vider la ligne
int age = sc.nextInt();
sc.nextLine();
String nom = sc.nextLine();
```

Pourquoi ? nextInt() ne consomme pas le caractère de fin de ligne, que nextLine() lit ensuite.

Console

```
Console c = System.console();    // null si lancé depuis
un IDE
String login = c.readLine("Login : ");
char[] mdp = c.readPassword("Mot de passe : ");
```

Structures de contrôle

- Les structures de contrôle utilisées en Java sont fortement inspirées de celles du langage C ou C++ :
  - if ; else ; while ; do ; for ; break ; goto

Branchement conditionnel (1/2)

- Si … Alors … Sinon

```
if (test)
      {
       action1_si_test_vrai;
       action2_si_test_vrai;
      }
else
      {
       action1_si_test_faux;
       action2_si_test_faux;
      }
```

- Le test est de la forme :

  a ; !a ; a & b ; a & !b ;

  x<y ; x == y; x != y ; …

Branchement conditionnel (2/2)

- Branchement selon un choix multiple :

```
switch (expression)
{
    case valeur_1 : actions; break;
    case valeur_2 : actions; break;
    case valeur_3 : actions; break;
    default : actions si valeur inconnue;
}
```

Boucle "Tant que"

- **while** (test)
  {
  actions;
  }

- **do**
  {
  actions;
  }
  while (test);

Boucle "Pour"

- int i;
  for(i=0; i<5; i++) // *ou* ***for***(*int i=0; i<5; i++*)
  {
  actions;
  }

Sorties de boucles

- L'instruction **break** permet de sortir d'une boucle avant sa fin "normale".
- L'instruction **continue** passe le reste de la boucle, mais n'en sort pas.
- L'instruction **goto label** subsiste encore. Le label doit alors être défini (et suivi de **:**).

Les tableaux

Un tableau contient un ensemble fini d'éléments d'un même type. Les éléments sont accessibles par un indice ; le premier élément possède l'indice 0.

En Java, les tableaux sont des objets dérivés de Object.

Ils peuvent utiliser des méthodes héritées comme equals() et getClass().

Déclaration et allocation

Deux syntaxes de déclaration sont possibles :

```
int tableau[] = new int[50];

int[] tableau = new int[50];
```

La déclaration et l'allocation peuvent être séparées :

```
int tab[];
tab = new int[50];
```

Passage d'un tableau a une méthode

Un tableau peut être passé comme paramètre d'une méthode.

Exemple :

```
public void afficher(String texte[]) {
    // ...
}
```

Tableaux multidimensionnels

Java ne supporte pas directement les tableaux multidimensionnels.

On utilise un tableau de tableaux.

Une paire de crochets est utilisée pour chaque dimension.

```
float tableau[][] = new float[10][10];
```

Initialisation automatique

Chaque élément du tableau est initialisé selon son type par l'instruction new :

0 pour les numériques

'\0' pour les caractères

false pour les booléens

null pour les chaînes de caractères et les autres objets

Initialisation explicite

```
int tableau[5] = {10, 20, 30, 40, 50};
int tableau[3][2] = {{5, 1}, {6, 2}, {7, 3}};
int tableau[] = {10, 20, 30, 40, 50};
int[][] tabEntiers = {{1, 2, 3, 4, 5, 6},{1, 2, 3, 4},
 {1, 2, 3, 4, 5, 6, 7, 8, 9}};
```

Le parcours d'un tableau

La variable length retourne le nombre d'éléments du tableau.

Il est alors possible d'utiliser une boucle pour itérer sur chacun des éléments.

Exemple :

```
for (int i = 0; i < tableau.length; i++) {
    // ...
}
```

Parcours avec for-each

```
for (type element_courant : type[] ) {
    // code à exécuter sur l'élément courant identifié
par element_courant
}
```

Exemple :

```
String[] chaines = {"element1","element2","element3"};
for (String chaine : chaines) {
    System.out.println(chaine);
}
```